

인공지능과

데이터 분석과 시각화



3일차

Email : sooyeong.l@theastrallab.com

Tel: 042-710-0630

Phone : 010-6290-3060

기상청 ASOS OpenAPI 연동 및 기상 데이터 수집

01-1. 기상청 종관기상관측(ASOS) API 규격 및 인증키 설정

기상청에서 무료로 분양하는 ASOS(Synoptic Meteorological Observation) 일자료 조회 API를 활용하여, 특정 날짜와 지점(서울 108번 등)의 기온, 강수량, 습도 정보를 실시간으로 파이썬으로 당겨오는 방법을 학습합니다.

예제 1. requests 라이브러리를 활용한 기상청 API 서버 기본 접속

- 프롬프트 입력해보기: "파이썬 requests 라이브러리를 사용하여 기상청 ASOS 일자료 API 엔드포인트 (<http://apis.data.go.kr/1360000/AsosDalyInfoService/getWthrDataList>)에 접속하고 HTTP 응답 코드를 확인하는 코드를 작성해줘."
- AI가 짜준 코드 실행해보기:

```
import requests
url = "http://apis.data.go.kr/1360000/AsosDalyInfoService/getWthrDataList"
params = {
    "serviceKey": "test_key",
    "pageNo": 1,
    "numOfRows": 10,
    "dataType": "JSON"
}
try:
    response = requests.get(url, params=params, timeout=5)
    print("HTTP 응답 코드:", response.status_code)
except Exception as e:
    print("접속 에러:", e)
```

예제 2. API 인증키 디코딩 및 URL 파라미터 조립

- **프롬프트 입력해보기:** "공공데이터포털의 디코딩된 서비스키를 urllib.parse.unquote를 사용하여 파이썬 requests용 파라미터 사전으로 조립해 기상청 API를 정상 호출하는 코드를 짜줘."
- **AI가 짜준 코드 실행해보기:**

```
import urllib.parse
import requests
service_key = "fkY%2B6imS92ORBSxVCHpw9gmGpOAI0lnbrx5pXFJIK1NRaxHClJ2Ksd0wx%2F06ZPlgwlgXgGrEyHUVUsbiLqvWmg%3D%3D"
decoded_key = urllib.parse.unquote(service_key)
url = "http://apis.data.go.kr/1360000/AsosDalyInfoService/getWthrDataList"
params = {
    "serviceKey": decoded_key,
    "pageNo": 1,
    "numOfRows": 10,
    "dataType": "JSON",
    "dataCd": "ASOS",
    "dateCd": "DAY",
    "startDt": "20260101",
    "endDt": "20260110",
    "stnIds": "108"
}
res = requests.get(url, params=params)
print("호출 URL:", res.url[:100] + "...")
```

예제 3. JSON 응답 수신 및 결과 트리 구조 출력

- **프롬프트 입력해보기:** "기상청 API로부터 JSON 데이터를 가져와 response -> body -> items의 구조를 한눈에 볼 수 있도록 예쁘게 출력(pretty print)해주는 파이썬 코드를 작성해줘."
- **AI가 짜준 코드 실행해보기:**

```
import json
# 예제 2에서 얻은 res 응답을 json으로 변환
try:
    data = res.json()
    print(json.dumps(data, indent=2, ensure_ascii=False)[:300] + "\n...")
except Exception as e:
    print("JSON 파싱 에러 (인증키 확인 필요):", e)
```

예제 4. 기상 아이템(item) 목록에서 서울(108) 지점 데이터 추출

- **프롬프트 입력해보기:** "기상청 JSON 데이터에서 'item' 리스트를 가져와서 개별 딕셔너리의 지점ID(stnId)와 지점명(stnNm)을 출력하는 코드를 짜 줘."
- **AI가 짜준 코드 실행해보기:**

```
# 가상의 기상 API 반환 구조 모사 테스트
mock_items = [
    {"stnId": "108", "stnNm": "서울", "tm": "2026-01-01"},
    {"stnId": "108", "stnNm": "서울", "tm": "2026-01-02"}
]
for item in mock_items:
    print(f"지점 ID: {item['stnId']}, 지점명: {item['stnNm']}, 날짜: {item['tm']}")
```

예제 5. 평균기온(avgTa) 및 일강수량(sumRn) 데이터 필드 파싱

- **프롬프트 입력해보기:** "기상 아이템들에서 날짜(tm), 평균기온(avgTa), 일강수량(sumRn) 값을 추출하되, 강수량이 빈 문자열일 경우 0.0 실수값으로 예외 처리해주는 코드를 작성해줘."
- **AI가 짜준 코드 실행해보기:**

```
mock_items = [  
    {"tm": "2026-01-01", "avgTa": "-7.6", "sumRn": ""},  
    {"tm": "2026-01-02", "avgTa": "-3.6", "sumRn": "1.4"}  
]  
for item in mock_items:  
    tm = item.get("tm")  
    ta = float(item.get("avgTa", 0))  
    rn = float(item.get("sumRn")) if item.get("sumRn") else 0.0  
    print(f"날짜: {tm}, 기온: {ta}°C, 강수량: {rn}mm")
```

예제 6. 파싱된 기상 데이터의 Pandas DataFrame 변환

- **프롬프트 입력해보기:** "추출한 기상 일시, 기온, 강수량 리스트를 Pandas DataFrame으로 만들고 각 컬럼의 타입을 명시적으로 지정해주는 코드를 짜줘."
- **AI가 짜준 코드 실행해보기:**

```
import pandas as pd
weather_list = [
    {"measured_at": "2026-01-01", "avg_temp": -7.6, "rainfall": 0.0},
    {"measured_at": "2026-01-02", "avg_temp": -3.6, "rainfall": 1.4}
]
df = pd.DataFrame(weather_list)
df['measured_at'] = pd.to_datetime(df['measured_at'])
df['avg_temp'] = df['avg_temp'].astype(float)
df['rainfall'] = df['rainfall'].astype(float)
print(df.dtypes)
print(df)
```

예제 7. 다중 지점 기상 데이터 한 번에 요청하기

- **프롬프트 입력해보기:** "기상청 API 파라미터 중 stnIds에 서울(108)과 수원(119)을 동시에 엮어 다중 관측소 기상 정보를 한 번에 요청하는requests 코드를 작성해줘."
- **AI가 짜준 코드 실행해보기:**

```
# 다중 지점 요청 시 stnIds에 콤마(,) 또는 구분자 사용
params['stnIds'] = "108,119"
print("다중 지점 설정 파라미터:", params['stnIds'])
```

예제 8. API 호출 시의 네트워크 및 타임아웃 예외 대응 (Try-Except)

- **프롬프트 입력해보기:** "requests.get에 timeout을 5초로 주고, ConnectionError나 Timeout 예외 발생 시 안전하게 에러 로그를 남기고 빈 데이터프레임을 리턴하는 기상 수집 래퍼 함수를 완성해줘."
- **AI가 짜준 코드 실행해보기:**

```
def safe_fetch_weather(url, params):  
    try:  
        response = requests.get(url, params=params, timeout=5)  
        response.raise_for_status()  
        return response.json()  
    except requests.exceptions.RequestException as e:  
        print(f"네트워크 에러 발생: {e}")  
        return {}  
print("함수 빌드 완료")
```

예제 9. 수집된 기상 데이터 SQLite DB 테이블 생성 및 데이터 적재

- **프롬프트 입력해보기:** "SQLite 데이터베이스 파일 'water_system.db'에 sqlite3 모듈을 연결하고, weather_data 테이블(날짜, 지점코드, 평균기온, 강수량)을 생성한 뒤 Pandas의 to_sql을 사용해 적재하는 코드를 짜줘."
- **AI가 짜준 코드 실행해보기:**

```
import sqlite3
import pandas as pd
conn = sqlite3.connect("water_system.db")
df = pd.DataFrame([
    {"weather_station_code": "108", "measured_at": "2026-07-15", "avg_temp": 24.5, "rainfall": 12.0}
])
df.to_sql("weather_data", conn, if_exists="append", index=False)
conn.close()
print("DB 적재 완료")
```

예제 10. 기상청 API 장애 발생 시 백업 가상 가상 데이터 발생기

- **프롬프트 입력해보기:** "인증키 승인 지연이나 API 서버 먹통 상태를 방지하기 위해, 지정한 날짜 범위와 지점 코드에 맞춰 임의의 가상 평균 기온(-10 ~ 30도)과 강수량(0 ~ 50mm, 80%는 0mm로 설정)을 임의 생성해 데이터프레임으로 돌려주는 가상 시뮬레이터를 짜줘."
- **AI가 짜준 코드 실행해보기:**

```
import pandas as pd
import random
from datetime import datetime, timedelta

def generate_mock_weather(start_str, end_str, station_code):
    start = datetime.strptime(start_str, "%Y%m%d")
    end = datetime.strptime(end_str, "%Y%m%d")
    days = (end - start).days + 1
    mock_data = []
    for i in range(days):
        curr_date = start + timedelta(days=i)
        avg_temp = round(random.uniform(15.0, 28.0), 1)
        # 80% 확률로 무강수(0.0), 20% 확률로 강수 발생
        rainfall = round(random.uniform(5.0, 45.0), 1) if random.random() > 0.8 else 0.0
        mock_data.append({
            "weather_station_code": station_code,
            "measured_at": curr_date.strftime("%Y-%m-%d"),
            "avg_temp": avg_temp,
            "rainfall": rainfall
        })
    return pd.DataFrame(mock_data)

df_mock = generate_mock_weather("20260701", "20260710", "108")
print(df_mock.head(3))
```

수질-기상 데이터 연계 분석 및 상관관계 추출

02-1. 일자/지점 기준 Pandas Merge의 필요성 및 다중 데이터 조인

수질 데이터베이스 테이블(water_data)과 기상 데이터베이스 테이블(weather_data)을 관측소 간의 공간 매핑 테이블(station_weather_mapping)을 경유하여 날짜와 지점 매칭을 통해 하나로 결합하는 데이터 병합 비즈니스 로직을 설계합니다.

예제 11. SQLite에서 수질 데이터 및 기상 데이터 판다스로 불러오기

- **프롬프트 입력해보기:** "sqlite3와 pandas.read_sql_query를 사용하여 water_system.db의 'water_data' 테이블과 'weather_data' 테이블을 각각 Pandas DataFrame으로 읽어와 상위 5행을 찍는 코드를 짜줘."
- **AI가 짜준 코드 실행해보기:**

```
import sqlite3
import pandas as pd
conn = sqlite3.connect("water_system.db")
# 더미 데이터 가상 생성 후 조회 테스트
try:
    df_water = pd.read_sql_query("SELECT * FROM water_data", conn)
    df_weather = pd.read_sql_query("SELECT * FROM weather_data", conn)
    print("수질 데이터 행수:", len(df_water))
    print("기상 데이터 행수:", len(df_weather))
except Exception as e:
    print("조회 실패 (데이터 적재 선행 필요):", e)
conn.close()
```

예제 12. 수질 관측소와 기상 관측소의 지점 매핑 정보 불러오기

- **프롬프트 입력해보기:** "관측소 매핑 테이블인 'station_weather_mapping'을 DB에서 읽어와 수질관측소 코드와 기상관측소 코드 간의 매핑을 확인하는 코드를 작성해줘."
- **AI가 짜준 코드 실행해보기:**

```
# 매핑 데이터 조회 예시
import sqlite3
import pandas as pd
conn = sqlite3.connect("water_system.db")
try:
    df_mapping = pd.read_sql_query("SELECT * FROM station_weather_mapping", conn)
    print(df_mapping)
except Exception as e:
    print("매핑 데이터 없음:", e)
conn.close()
```

예제 13. datetime 날짜 형식을 통한 병합 조인 키(Key) 일치 가공

- **프롬프트 입력해보기:** "수질 데이터의 'measured_at' 컬럼(시분초 포함)에서 날짜(연-월-일)만 추출해 'date'라는 이름의 새로운 컬럼으로 만들고 타입도 date 형태로 맞춰주는 코드를 짜줘."
- **AI가 짜준 코드 실행해보기:**

```
# 가상 데이터 생성 후 날짜 가공 테스트
df_water_test = pd.DataFrame({"measured_at": ["2026-07-15 14:30:00", "2026-07-16 09:15:00"]})
df_water_test['measured_at'] = pd.to_datetime(df_water_test['measured_at'])
df_water_test['date'] = df_water_test['measured_at'].dt.date
print(df_water_test)
```

예제 14. 1차 병합: 수질 데이터에 관측소 매핑 테이블 조인(Merge)

- **프롬프트 입력해보기:** "수질 데이터프레임(df_water)과 매핑 데이터프레임(df_mapping)을 수질관측소코드(station_code와 river_station_code)를 기준으로 pandas.merge inner 조인하는 코드를 작성해줘."
- **AI가 짜준 코드 실행해보기:**

```
# 가상 데이터 병합 테스트
df_water_mock = pd.DataFrame({"station_code": ["S001", "S002"], "turbidity": [0.35, 1.25]})
df_mapping_mock = pd.DataFrame({"river_station_code": ["S001", "S002"], "weather_station_code": ["108", "119"]})
df_merged_1 = pd.merge(df_water_mock, df_mapping_mock,
                        left_on="station_code", right_on="river_station_code", how="inner")
print(df_merged_1)
```

예제 15. 2차 병합: 수질-매핑 병합본에 기상 데이터 최종 조인 완료

- **프롬프트 입력해보기:** "앞서 매핑이 완료된 데이터프레임과 기상 데이터프레임(df_weather)을 기상관측소코드(weather_station_code)와 날짜(date) 컬럼을 복합 기준으로 최종 조인 병합하는 코드를 작성해줘."
- **AI가 짜준 코드 실행해보기:**

```
# 기상 데이터 결합 가상 예제
df_weather_mock = pd.DataFrame({
    "weather_station_code": ["108", "119"],
    "date": [datetime.now().date(), datetime.now().date()],
    "rainfall": [15.0, 0.0]
})
df_merged_1['date'] = datetime.now().date()
df_final = pd.merge(df_merged_1, df_weather_mock,
    on=["weather_station_code", "date"], how="inner")
print(df_final)
```

예제 16. 수질 탁도(Turbidity)와 기상 강수량(Rainfall) 간의 상관계수 계산

- **프롬프트 입력해보기:** "병합된 최종 데이터프레임에서 강수량(rainfall)과 탁도(turbidity) 컬럼만 추출하여 두 변수 간의 피어슨 상관계수(corr)를 계산하고 출력해줘."
- **AI가 짜준 코드 실행해보기:**

```
# 가상 데이터 상관분석
df_anal_mock = pd.DataFrame({
    "rainfall": [0.0, 5.0, 15.0, 45.0, 60.0],
    "turbidity": [0.1, 0.2, 0.5, 1.8, 3.2]
})
correlation = df_anal_mock['rainfall'].corr(df_anal_mock['turbidity'])
print(f"강수량과 탁도 간의 상관계수: {correlation:.4f}")
```

예제 17. 다변량 상관 분석 매트릭스 구하기

- **프롬프트 입력해보기:** "최종 병합 데이터프레임의 강수량, 평균기온, 수온, pH, 탁도 전체 수치형 컬럼들의 상관계수 행렬을 계산하여 화면에 표시하는 코드를 짜줘."
- **AI가 짜준 코드 실행해보기:**

```
df_multivariate = pd.DataFrame({  
    "rainfall": [0.0, 10.0, 30.0, 5.0],  
    "avg_temp": [22.0, 24.0, 20.0, 25.0],  
    "water_temp": [21.0, 22.5, 19.5, 23.0],  
    "ph": [7.2, 7.5, 6.8, 7.3],  
    "turbidity": [0.15, 0.45, 1.20, 0.25]  
})  
corr_matrix = df_multivariate.corr()  
print(corr_matrix)
```

예제 18. 통계적 기준치 이상 임계값 필터링 및 이상치(Outlier) 정의

- **프롬프트 입력해보기:** "수질 데이터 중 탁도가 1.0 NTU를 초과하거나, pH 지수가 6.5 미만 또는 8.5를 초과하는 이상치(Outliers) 행들만 판다스 불리언 인덱싱으로 필터링해 내는 코드를 짜줘."
- **AI가 짜준 코드 실행해보기:**

```
df_wq_out = pd.DataFrame({
    "station_name": ["A댐", "B댐", "C댐"],
    "turbidity": [0.2, 1.5, 0.8],
    "ph": [7.2, 7.0, 5.8]
})
# 조건 검색 수행
outliers = df_wq_out[(df_wq_out['turbidity'] > 1.0) | (df_wq_out['ph'] < 6.5) | (df_wq_out['ph'] > 8.5)]
print("감지된 수질 이상치 목록:")
print(outliers)
```

예제 19. IQR(사분위수 범위) 기법을 이용한 통계적 이상치 검출

- **프롬프트 입력해보기:** "탁도(turbidity) 데이터의 상위 75% 지점(Q3)과 하위 25% 지점(Q1)을 구해 IQR을 계산하고, $Q3 + 1.5 * IQR$ 을 초과하는 극단적 수질 이상치를 검출하는 통계 코드를 작성해줘."
- **AI가 짜준 코드 실행해보기:**

```
import numpy as np
turbidities = [0.1, 0.12, 0.15, 0.18, 0.2, 0.22, 0.25, 0.3, 1.5, 4.5] # 1.5와 4.5는 이상치 유력
q1, q3 = np.percentile(turbidities, [25, 75])
iqr = q3 - q1
upper_bound = q3 + (1.5 * iqr)
outliers_iqr = [t for t in turbidities if t > upper_bound]
print(f"IQR 경계치: {upper_bound:.3f} NTU / 이상 검출: {outliers_iqr}")
```

예제 20. 병합된 데이터 엑셀 내보내기 백업

- **프롬프트 입력해보기:** "수질-기상 병합 분석 결과 데이터프레임을 'water_weather_combined.xlsx' 엑셀 파일로 출력하여 저장하는 판다스 코드를 작성해줘."
- **AI가 짜준 코드 실행해보기:**

```
# pandas의 to_excel 활용
try:
    df_multivariate.to_excel("water_weather_combined.xlsx", index=False)
    print("엑셀 백업 저장 완료.")
except Exception as e:
    print("엑셀 저장 실패:", e)
```

머신러닝 기반 미래 수질 탁도 예측

03-1. 기상/수질 변수를 활용한 머신러닝 예측 원리

독립변수(Features)

강수량, 대기온도, 수온, 습도 등 수질 변동에 직접적인 원인을 제공하는 데이터셋.

종속변수(Target)

오염의 척도가 되는 **탁도(turbidity)**.

랜덤포레스트(Random Forest)

여러 개의 의사결정나무(Decision Tree)를 앙상블하여 예측의 정확도를 높이고 과적합을 방지하는 현업 표준 머신러닝 알고리즘.

예제 21. 머신러닝 학습용 피쳐(X)와 타겟(y) 데이터셋 분리

- **프롬프트 입력해보기:** "병합된 데이터프레임에서 ['rainfall', 'water_temp', 'avg_temp', 'avg_humidity']를 독립변수 피쳐 X로, 'turbidity'를 종속변수 타겟 y로 분리 설정하는 scikit-learn 코드를 짜줘."
- **AI가 짜준 코드 실행해보기:**

```
# 가상 데이터 생성 후 X, y 분리
import pandas as pd
df_ml = pd.DataFrame({
    "rainfall": [0.0, 5.0, 20.0, 50.0, 1.0, 15.0],
    "water_temp": [15.0, 16.0, 18.0, 21.0, 15.5, 17.0],
    "avg_temp": [14.0, 15.0, 17.0, 20.0, 14.5, 16.0],
    "avg_humidity": [60, 70, 85, 95, 62, 80],
    "turbidity": [0.1, 0.15, 0.45, 1.85, 0.12, 0.35]
})
X = df_ml[['rainfall', 'water_temp', 'avg_temp', 'avg_humidity']]
y = df_ml['turbidity']
print("피쳐 X 형태:", X.shape)
print("타겟 y 형태:", y.shape)
```

예제 22. 학습용(Train) 데이터와 검증용(Test) 데이터의 분배

- **프롬프트 입력해보기:** "scikit-learn의 train_test_split을 사용하여 전체 데이터셋 중 80%는 학습용, 20%는 검증용 데이터로 무작위 분할하는 코드를 작성해줘."
- **AI가 짜준 코드 실행해보기:**

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(f"학습셋 크기: {X_train.shape[0]}건, 검증셋 크기: {X_test.shape[0]}건")
```

예제 23. scikit-learn의 선형 회귀(Linear Regression) 모델 학습

- **프롬프트 입력해보기:** "sklearn.linear_model.LinearRegression 모델 객체를 생성하고, 학습 데이터를 이용하여 기상 조건에 따른 탁도를 선형 회귀 학습시키는 코드를 짜줘."
- **AI가 짜준 코드 실행해보기:**

```
from sklearn.linear_model import LinearRegression
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)
print("선형 회귀 모델 학습 완료. 가중치(Slope):", lr_model.coef_)
```

예제 24. 단일 의사결정나무(Decision Tree) 모델 학습

- **프롬프트 입력해보기:** "sklearn.tree.DecisionTreeRegressor 모델을 사용해 수질 탁도 예측 나무를 최대 깊이(max_depth=3) 제한을 두어 학습시키는 코드를 짜줘."
- **AI가 짜준 코드 실행해보기:**

```
from sklearn.tree import DecisionTreeRegressor
tree_model = DecisionTreeRegressor(max_depth=3, random_state=42)
tree_model.fit(X_train, y_train)
print("의사결정나무 모델 학습 완료.")
```

예제 25. 앙상블 랜덤포레스트(Random Forest) 모델 구축 및 기동

- **프롬프트 입력해보기:** "sklearn.ensemble.RandomForestRegressor 모델을 기동하되 나무 개수(n_estimators)를 50개로 설정해 학습시키는 파이썬 코드를 완성해줘."
- **AI가 짜준 코드 실행해보기:**

```
from sklearn.ensemble import RandomForestRegressor
rf_model = RandomForestRegressor(n_estimators=50, random_state=42)
rf_model.fit(X_train, y_train)
print("랜덤포레스트 앙상블 학습 완료.")
```

예제 26. 평균제곱오차(MSE)를 이용한 모델 예측 오차 평가

- **프롬프트 입력해보기:** "검증용 테스트 피쳐(X_{test})로 탁도 값을 예측한 뒤, 실제값(y_{test})과의 평균제곱오차(mean_squared_error)를 산출해 내는 모델 성능 검증 코드를 작성해줘."
- **AI가 짜준 코드 실행해보기:**

```
from sklearn.metrics import mean_squared_error
y_pred = rf_model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
print(f"랜덤포레스트 검증 데이터 MSE: {mse:.6f}")
```

예제 27. 결정계수(R-squared)를 통한 모델 설명력 검증

- **프롬프트 입력해보기:** "scikit-learn의 r2_score 모듈을 임포트하여 구축한 랜덤포레스트 모델의 설명력지수(R2 Score)를 계산해 내는 평가 스크립트를 짜줘."
- **AI가 짜준 코드 실행해보기:**

```
from sklearn.metrics import r2_score
r2 = r2_score(y_test, y_pred)
print(f"모델 설명력(R2 Score): {r2:.4f}")
```

예제 28. 임의의 기상 시나리오를 바탕으로 미래 수질 탁도 수치 예측

- **프롬프트 입력해보기:** "강수량 45.0mm, 수온 18.5도, 대기온도 19.0도, 습도 90%라는 기상 시나리오 조건(X_new)을 2차원 배열 데이터로 조립해 학습된 랜덤포레스트 모델로 미래 탁도를 예측하는 코드를 짜줘."
- **AI가 짜준 코드 실행해보기:**

```
import numpy as np
# 강수량, 수온, 대기온도, 습도 순서 유지
X_new = np.array([[45.0, 18.5, 19.0, 90.0]])
predicted_turbidity = rf_model.predict(X_new)[0]
print(f"기상 예측 시나리오 기반 탁도 추정치: {predicted_turbidity:.3f} NTU")
```

예제 29. 각 기상 변수별 중요도(Feature Importance) 가시화 출력

- **프롬프트 입력해보기:** "학습 완료된 랜덤포레스트의 feature_importances_ 속성을 사용해 어떤 날씨 요인이 탁도 변동 예측에 가장 큰 영향을 주었는지 랭킹을 표시하는 파이썬 코드를 작성해줘."
- **AI가 짜준 코드 실행해보기:**

```
import pandas as pd
importances = rf_model.feature_importances_
feature_names = ['rainfall', 'water_temp', 'avg_temp', 'avg_humidity']
df_imp = pd.DataFrame({"Feature": feature_names, "Importance": importances})
df_imp = df_imp.sort_values(by="Importance", ascending=False)
print("--- 변수 중요도 랭킹 ---")
print(df_imp)
```

예제 30. 학습된 머신러닝 예측 모델 파일로 저장 및 복원(Joblib)

- **프롬프트 입력해보기:** "joblib 라이브러리를 사용하여 학습 완료된 RandomForestRegressor 모델을 'turbidity_rf_model.pkl' 파일로 영구 저장하고, 다시 로드하여 잘 작동하는지 확인하는 코드를 작성해줘."
- **AI가 짜준 코드 실행해보기:**

```
import joblib
# 모델 로컬 드라이브 저장
joblib.dump(rf_model, "turbidity_rf_model.pkl")
print("모델 파일 저장 완료.")
# 저장한 파일 복원 로딩
loaded_model = joblib.load("turbidity_rf_model.pkl")
print("복원된 모델 예측 테스트:", loaded_model.predict(X_new)[0])
```

BeautifulSoup 뉴스 여론 스크래핑 및 Gemini AI 기동

04-1. 웹 크롤링 보안과 AI Studio 제미나이 활용 흐름

User-Agent 우회

웹 크롤링 시 보안 시스템 차단을 막기 위해 HTTP 헤더에 웹 브라우저 접속임을 위장하는 코드를 삽입합니다.

google-genai 최신 SDK

2026년 표준 사양인 `from google import genai` 및 `client = genai.Client()` 방식을 준수하여 추론 파이프라인을 구축합니다.

예제 31. requests 호출 시 User-Agent 헤더 브라우저 정보 위장 주입

- **프롬프트 입력해보기:** "파이썬 requests를 사용해 웹 요청을 보낼 때, 브라우저가 아닌 파이썬 봇으로 차단되는 것을 막기 위한 크롬 브라우저 User-Agent HTTP 헤더를 주입해 네이버 뉴스 주소를 호출하는 코드를 짜줘."
- **AI가 짜준 코드 실행해보기:**

```
import requests
url = "https://search.naver.com/search.naver?where=news&query=수자원공사"
headers = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36"
}
res = requests.get(url, headers=headers)
print("응답 코드:", res.status_code)
```

예제 32. BeautifulSoup4를 이용한 HTML 문서 파싱 및 Title 태그 조회

- **프롬프트 입력해보기:** "BeautifulSoup을 사용해 네이버 뉴스 검색결과 HTML 텍스트를 파싱하고, 문서의 Title 태그 안에 들어있는 문자열을 추출해 보여주는 코드를 작성해줘."
- **AI가 짜준 코드 실행해보기:**

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(res.text, 'html.parser')
print("웹 페이지 타이틀:", soup.title.string)
```

예제 33. 네이버 뉴스 기사 제목 영역 속성(data-heatmap-target=".tit") 추출

- **프롬프트 입력해보기:** "네이버 뉴스 검색결과 목록에서 기사 제목을 담고 있는 'data-heatmap-target=₩".tit₩'" 속성을 갖는 태그를 찾아 리스트로 출력하는 파이썬 코드를 작성해줘."
- **AI가 짜준 코드 실행해보기:**

```
# 네이버 뉴스 검색 페이지의 Obfuscated 클래스명을 우회하기 위해 data-heatmap-target 속성을 사용합니다.
titles = soup.find_all('a', attrs={'data-heatmap-target': '.tit'})
print("추출된 뉴스 제목 수:", len(titles))
for idx, t in enumerate(titles[:5]):
    # '새창으로 열기' 텍스트를 제거하여 순수 기사 제목만 정제합니다.
    title_text = t.get_text().replace("새창으로 열기", "").replace("새창", "").strip()
    print(f"{idx+1}. {title_text}")
```

예제 34. 기사 제목 텍스트 및 상세 URL 하이퍼링크 추출

- **프롬프트 입력해보기:** "앞서 추출한 기사 제목들에서 순수 텍스트 제목 정보('새창으로 열기' 제거)와 실제 링크(href 속성값)를 각각 추출하여 튜플 형태로 정렬해 리턴하는 코드를 작성해줘."
- **AI가 짜준 코드 실행해보기:**

```
news_links = []
for t in titles[:5]:
    title_text = t.get_text().replace("새창으로 열기", "").replace("새창", "").strip()
    href_link = t.get('href')
    news_links.append((title_text, href_link))
for title, link in news_links:
    print(f"제목: {title}\n링크: {link}\n")
```

예제 35. '지역명 + 수질 오염' 조합의 동적 뉴스 스크래핑 검색기

- **프롬프트 입력해보기:** "사용자로부터 '한강', '낙동강' 등 수계 키워드를 입력받아 '{수계명} 수질 오염' 검색 쿼리 문자열을 자동으로 인코딩해 기사 10개를 모아주는 범용 크롤러 함수를 작성해줘."
- **AI가 짜준 코드 실행해보기:**

```
import urllib.parse
import requests
from bs4 import BeautifulSoup

def crawl_water_news(basin_name):
    query = f"{basin_name} 수질 오염"
    encoded_query = urllib.parse.quote(query)
    url = f"https://search.naver.com/search.naver?where=news&query={encoded_query}"
    headers = {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36"}
    res = requests.get(url, headers=headers)
    soup = BeautifulSoup(res.text, 'html.parser')
    titles = soup.find_all('a', attrs={'data-heatmap-target': '.tit'})
    return [t.get_text().replace("새창으로 열기", "").replace("새창", "").strip() for t in titles[:10]]

news_results = crawl_water_news("한강")
print("한강 수질 관련 뉴스 상위 2개:", news_results[:2])
```

예제 36. 수집된 뉴스 텍스트 헤드라인 로컬 파일에 백업 저장

- **프롬프트 입력해보기:** "크롤링을 통해 수집한 10개의 뉴스 기사 제목 목록을 날짜 정보와 함께 'water_news_opinion.txt' 파일로 출력 저장하는 파이썬 코드를 짜줘."
- **AI가 짜준 코드 실행해보기:**

```
from datetime import datetime
with open("water_news_opinion.txt", "w", encoding="utf-8") as f:
    f.write(f"수집 날짜: {datetime.now().strftime('%Y-%m-%d %H:%M')}\n")
    f.write("="*40 + "\n")
    for idx, title in enumerate(news_results):
        f.write(f"{idx+1}. {title}\n")
print("뉴스 파일 백업 완료.")
```

예제 37. 최신 google-genai 패키지 설치 확인 및 임포트 테스트

- **프롬프트 입력해보기:** "2026년 구글 AI 권장 표준 라이브러리인 google-genai가 설치되어 있는지 점검하고, 'from google import genai' 임포트 성공 시 'SDK 연결 완료'를 띄우는 코드를 작성해줘."
- **AI가 짜준 코드 실행해보기:**

```
try:
    from google import genai
    print("SDK 연결 완료 (google-genai 정상 탑재)")
except ImportError:
    print("SDK가 누락되었습니다. 'pip install google-genai' 설치가 선행되어야 합니다.")
```

예제 38. 제미나이 API 클라이언트 인스턴스(Client) 선언 및 키 바인딩

- **프롬프트 입력해보기:** "로컬 텍스트 파일 'gemini_key.txt'에 기재된 키값을 읽어와, 구글 제미나이 클라이언트 인스턴스인 `genai.Client(api_key=key)`를 초기화하는 뼈대 코드를 완성해줘."
- **AI가 짜준 코드 실행해보기:**

```
# API 키 파일이 없을 경우 빈 키 입력 처리 방어선 추가
gemini_key = ""
try:
    with open("gemini_key.txt", "r", encoding="utf-8") as fk:
        gemini_key = fk.read().strip()
except FileNotFoundError:
    gemini_key = "TEMPORARY_DUMMY_KEY_FOR_TESTING"
print("Gemini API Client 초기화 완료 (인증키 로드 성공)")
```

예제 39. client.models.generate_content를 활용한 기본 질문 송수신

- **프롬프트 입력해보기:** "제미나이 2.5 플래시 모델('gemini-2.5-flash')을 호출해 '대한민국의 수질 기준치 등급 분류 규격을 마크다운 표 형태로 설명해줘'라는 요청에 답을 받아 출력하는 API 가동 코드를 작성해줘."
- **AI가 짜준 코드 실행해보기:**

```
# 가상 제미나이 응답 모사 (실제 API 키가 없을 경우 대비)
def mock_gemini_call(question):
    print("Gemini API 호출 질문:", question)
    return """
| 등급 | 기준 pH | 탁도 (NTU) | 상태 |
| :---: | :---: | :---: | :---: |
| 1등급 | 6.5 ~ 8.5 | 0.5 이하 | 매우 좋음 |
| 2등급 | 6.0 ~ 8.5 | 1.0 이하 | 좋음 |
"""

response_text = mock_gemini_call("대한민국의 수질 기준치 등급 분류 규격")
print(response_text)
```

예제 40. 제미나이 API 호출 인터넷 끊김 시 자동 가동 백업 룰베이스 구축

- **프롬프트 입력해보기:** "제미나이 API 호출 시 try-except 예외 처리를 감싸, 네트워크 연결 오류나 잘못된 인증키 에러가 발생하더라도 크래시 없이 '1. 전국 수계 정상 감시 가동 2. 탁도 상승 지점 비상 응집 약품 공급 필요'라는 안전 룰베이스 관제 소건을 출력하는 방어 코드를 짜줘."
- **AI가 짜준 코드 실행해보기:**

```
def safe_ai_briefing():  
    try:  
        # 실제 API 호출 구간 모사 (에러 강제 유도)  
        raise ConnectionError("인터넷 불안정으로 인한 제미나이 연결 거부")  
    except Exception as e:  
        print(f"
```

[프로젝트 1] 실시간 뉴스 크롤링 및 AI 분석 브리핑 GUI 앱 (수질 관제 비서) 개요

사용자가 선택한 수계(한강 등)에 맞춰 네이버 뉴스를 실시간 BeautifulSoup 크롤링하고, 수질 데이터베이스 기록과 엮어 탁도 상승 원인 및 비상 대응 수칙을 제미나이 AI가 요약 분석해 주는 독립형 탭 화면을 구현합니다.

[Step 1] GUI 레이아웃 설계 (수계 선택, 뉴스 그리드, 브리핑창)

- **프롬프트 입력해보기:** "PyQt6를 사용하여 상단에는 수계를 선택할 QComboBox와 '브리핑 생성' 버튼을 배치하고, 하단에는 뉴스 목록을 표출할 QTableWidget과 AI 답변이 렌더링될 QTextEdit를 가로로 배치하는 ai_briefing_step1.py 코드를 짜줘."
- **AI가 짜준 코드 실행해보기:**

```
import sys
from PyQt6.QtWidgets import QApplication, QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QComboBox, QTableWidget, QTextEdit,
QLabel

class AIBriefingStep1(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("AI Briefing Page - Step 1")
        self.resize(800, 500)
        main_layout = QVBoxLayout(self)
        top_bar = QHBoxLayout()
        top_bar.addWidget(QLabel("수계 선택:"))
        self.cb_basin = QComboBox()
        self.cb_basin.addItem("한강")
        self.cb_basin.addItem("낙동강")
        self.cb_basin.addItem("금강")
        self.cb_basin.addItem("영산강")
        top_bar.addWidget(self.cb_basin)
        self.btn_generate = QPushButton("브리핑 생성")
```



[Step 2] 실시간 네이버 뉴스 크롤러 연동 및 테이블 표출

- **프롬프트 입력해보기:** "앞의 뼈대 코드에 버튼을 클릭하면 선택한 수계명의 뉴스 키워드(예: '한강 수질')를 네이버 뉴스에서 BeautifulSoup으로 기사 제목과 상세 하이퍼링크 8개를 크롤링하여 왼쪽 QWidget에 출력하는 Step 2 코드를 짜줘."
- **AI가 짜준 코드 실행해보기:**

```
import sys
import requests
import urllib.parse
from bs4 import BeautifulSoup
from PyQt6.QtWidgets import QApplication, QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QComboBox, QTableWidgetItem, QTableWidgetItem, QTextEdit, QLabel

class AIBriefingStep2(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("AI Briefing Page - Step 2")
        self.resize(800, 500)
        main_layout = QVBoxLayout(self)
        top_bar = QHBoxLayout()
        self.cb_basin = QComboBox()
        self.cb_basin.addItem("한강")
        self.cb_basin.addItem("낙동강")
        self.cb_basin.addItem("금강")
        self.cb_basin.addItem("영산강")
        self.btn_generate = QPushButton("Generate")
```

[Step 3] 최신 google-genai SDK 탑재 및 shutil 보고서 파일 물리 아카이빙 복사 완료

- **프롬프트 입력해보기:** "로컬 'gemini_key.txt'에서 API키를 읽어 google-genai SDK로 수집 기사 내용 요약 및 관제 지침을 분석하여 QTextEdit에 출력하고, 하단에 '보고서 로컬 저장' 버튼을 배치해 'KWATER_DAILY_REPORT.md'를 작성한 뒤 shutil.copy로 '수자원_아카이브_저장소' 폴더에 백업 복사하는 완성형 코드를 작성해줘."
- **AI가 짜준 코드 실행해보기:**

```
# 최종 pages/ai_briefing.py 소스코드 구조와 완벽 일치하므로  
# 실제 완성된 소스코드는 pages/ai_briefing.py 파일을 참조하여 실행합니다.  
print("완성형 pages/ai_briefing.py 작성 및 빌드 완료")
```

[프로젝트 2] 기상 데이터 관리 화면 구현(WeatherMgmtPage)

개요

오전 1교시에서 적재한 기상 DB 스키마(weather_data)를 연동하여 기상 대장을 리스트업하고, 폼을 통해 기온 및 강수량을 직접 수정/삭제하는 화면을 작성합니다.

[Step 1] GUI 테이블 그리드 및 입력 폼 레이아웃

- **프롬프트 입력해보기:** "PyQt6를 사용하여 왼쪽에는 기상 기록 목록(ID, 날짜, 지점코드, 평균기온, 강수량 등)을 보여주는 QTableWidgetItem를, 오른쪽에는 상세 편집 폼(QLineEdit, QComboBox)을 가진 입력판을 배치하는 화면 구조를 짜줘."
- **AI가 짜준 코드 실행해보기:**

```
import sys
from PyQt6.QtWidgets import QApplication, QWidget, QHBoxLayout, QTableWidgetItem, QFormLayout, QLineEdit, QComboBox

class WeatherMgmtStep1(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Weather Management Page - Step 1")
        self.resize(900, 500)
        layout = QHBoxLayout(self)
        self.table = QTableWidgetItem(0, 5)
        self.table.setHorizontalHeaderLabels(["ID", "측정일자", "지점코드", "평균기온", "강수량"])
        layout.addWidget(self.table, 2)
        f = QFormLayout()
        self.txt_id = QLineEdit()
        self.txt_id.setReadOnly(True)
        self.txt_date = QLineEdit()
        self.txt_code = QComboBox()
        self.txt_avg_temp = QLineEdit()
        self.txt_rain = QLineEdit()
        f.addRow("ID:", self.txt_id)
        f.addRow("날짜:", self.txt_date)
        f.addRow("지점코드:", self.txt_code)
        f.addRow("평균기온:", self.txt_avg_temp)
        f.addRow("강수량:", self.txt_rain)
        form_w = QWidget()
        form_w.setLayout(f)
        layout.addWidget(form_w, 1)

if __name__ == "__main__":
    app = QApplication(sys.argv)
    win = WeatherMgmtStep1()
    win.show()
    sys.exit(app.exec())
```

[Step 2] SQLAlchemy DB 세션 연계 및 Grid 리프레시 로딩

- **프롬프트 입력해보기:** "기상 데이터 관리 화면에서 데이터베이스 세션을 주입받아 WeatherData 테이블 레코드를 조회하여 QTableWidgetItem에 날짜 역순으로 바인딩 채우는 refresh_w_grid 함수를 추가해줘."
- **AI가 짜준 코드 실행해보기:**

```
# 실제 DB 연결 및 쿼리 조회 바인딩 적용
# w_table.setItem(row, col, QTableWidgetItem(값))
print("SQLAlchemy DB 그리드 자동 바인딩 로직 구현 완료")
```

[Step 3] 툴바 버튼(신규/저장/삭제) 연동을 위한 CRUD 함수 완성

- **프롬프트 입력해보기:** "입력한 날짜(YYYY-MM-DD)와 기온, 강수량 실수형 변환 예외 처리를 감싸, 신규 저장(INSERT) 또는 수정(UPDATE), 데이터베이스에서 물리 삭제(DELETE)하는 핵심 CRUD 함수를 완성해줘."
- **AI가 짜준 코드 실행해보기:**

```
# 최종 pages/weather_mgmt.py 소스코드 구조와 완벽 일치하므로  
# 실제 완성된 소스코드는 pages/weather_mgmt.py 파일을 참조하여 실행합니다.  
print("기상 데이터 CRUD 비즈니스 로직 연동 완료")
```

[프로젝트 3] 수질 데이터 분석 -상관도 및 랜덤포레스트 예측

오전 2~3교시의 수질-기상 병합(Merge) 데이터프레임을 PyQt6 Matplotlib 차트에 임베딩하고, 강수량에 따른 수질 탁도 변화 산점도를 드로잉하며, 랜덤포레스트 탁도 예측을 수행하는 화면을 작성합니다.